


# … fabriquer mes logarithmes comme on fabrique des épingles.

*… to manufacture my logarithms as one manufactures pins.*

Gaspard de Prony, *Notice sur les grandes tables logarithmiques…*, 1824. English: lecturer

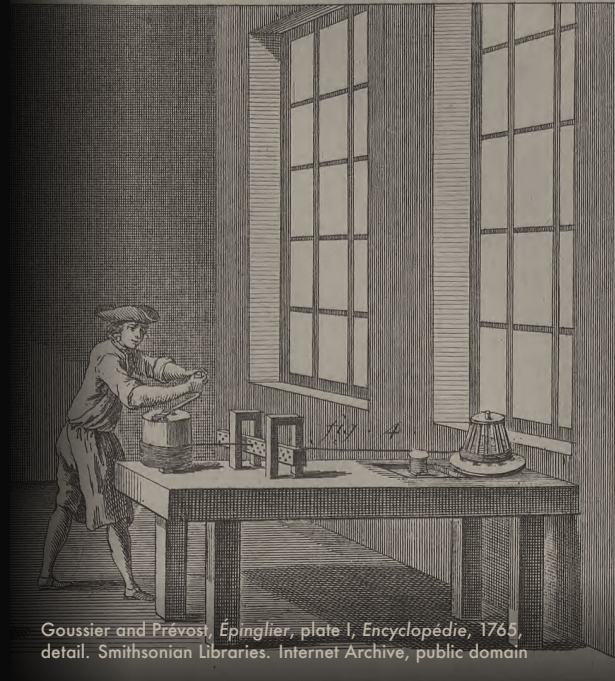
Goussier and Defehrt, *Épinglier*, plate II, *Encyclopédie*, 1765, detail. Smithsonian Libraries. Internet Archive, public domain

# Adam Smith

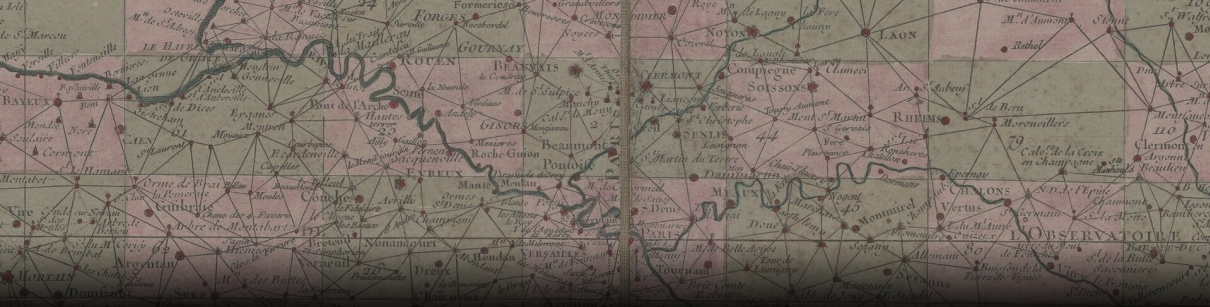
1723 to 1790

"One man draws out the wire; another straights it; a third cuts it; a fourth points it; a fifth grinds it at the top for receiving the head… Those ten persons, therefore, could make among them upwards of forty-eight thousand pins in a day."

Adam Smith, *The Wealth of Nations*, 1776, Book I, chapter 1



Goussier and Prévost, *Épinglier*, plate I, *Encyclopédie*, 1765, detail. Smithsonian Libraries. Internet Archive, public domain




# Every parcel in France

The French Revolution, 1790 to 1793

The Revolution taxes land, so the area of every parcel has to be computed. Land is measured in triangles, computed with tables of logarithms.

In 1793 Gaspard de Prony, director of the Bureau du cadastre in Paris, is ordered to compute new tables.

Maraldi and Cassini de Thury, map of the principal triangles of France, 1744, detail. BnF. Wikimedia Commons, public domain

# Three tiers

Bureau du cadastre, 1793 to 1796

Five or six mathematicians choose the formulas and pass them down.



The computers of the Harvard College Observatory, about 1890.
Wikimedia Commons, public domain

# Three tiers

Bureau du cadastre, 1793 to 1796

Five or six mathematicians choose the formulas and pass them down.

Seven or eight analysts turn them into the first values of each page and pass the pages down.



The computers of the Harvard College Observatory, about 1890.
Wikimedia Commons, public domain

# Three tiers

Bureau du cadastre, 1793 to 1796

Five or six mathematicians choose the formulas and pass them down.

Seven or eight analysts turn them into the first values of each page and pass the pages down.

About twenty computers fill in each page, using only additions and subtractions.

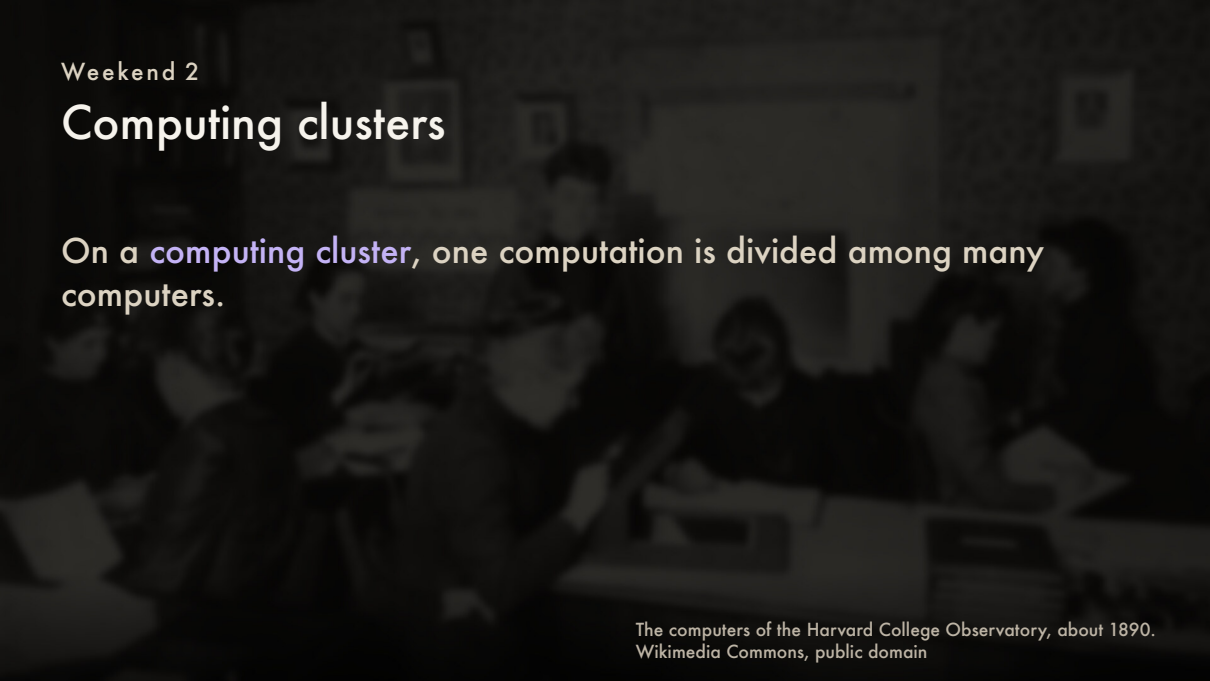


The computers of the Harvard College Observatory, about 1890.
Wikimedia Commons, public domain

Weekend 2

# Computing clusters

On a computing cluster, one computation is divided among many computers.



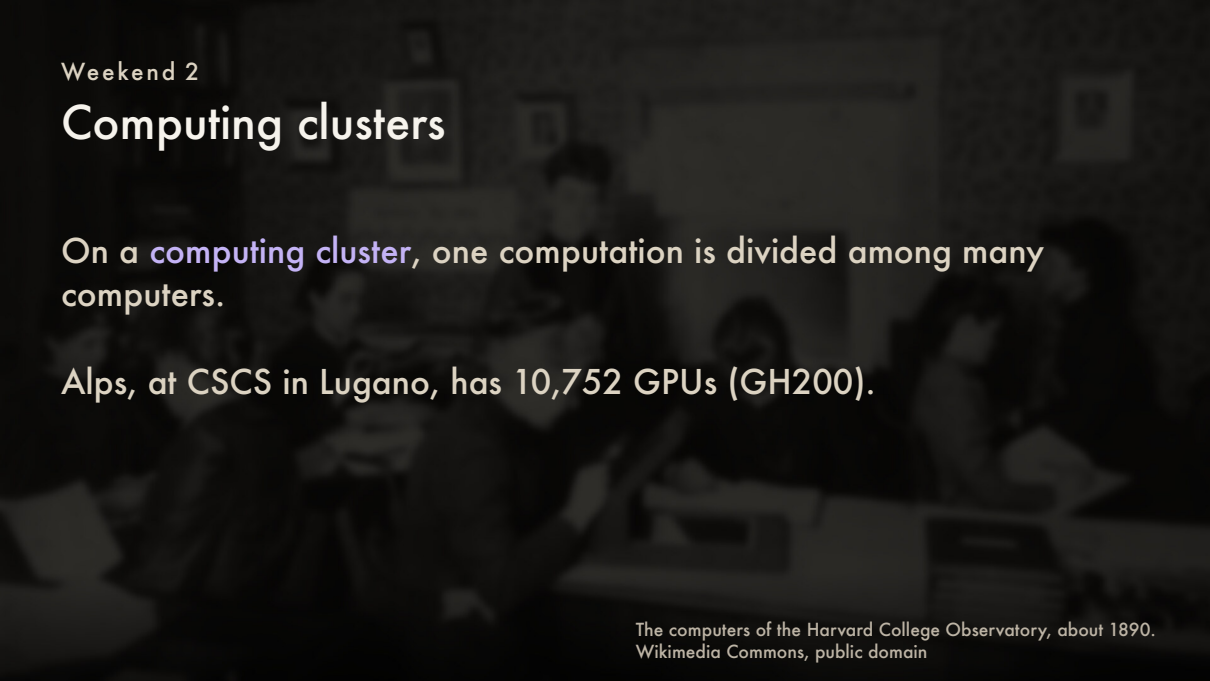
The computers of the Harvard College Observatory, about 1890.
Wikimedia Commons, public domain

Weekend 2

# Computing clusters

On a computing cluster, one computation is divided among many computers.

Alps, at CSCS in Lugano, has 10,752 GPUs (GH200).



The computers of the Harvard College Observatory, about 1890.
Wikimedia Commons, public domain

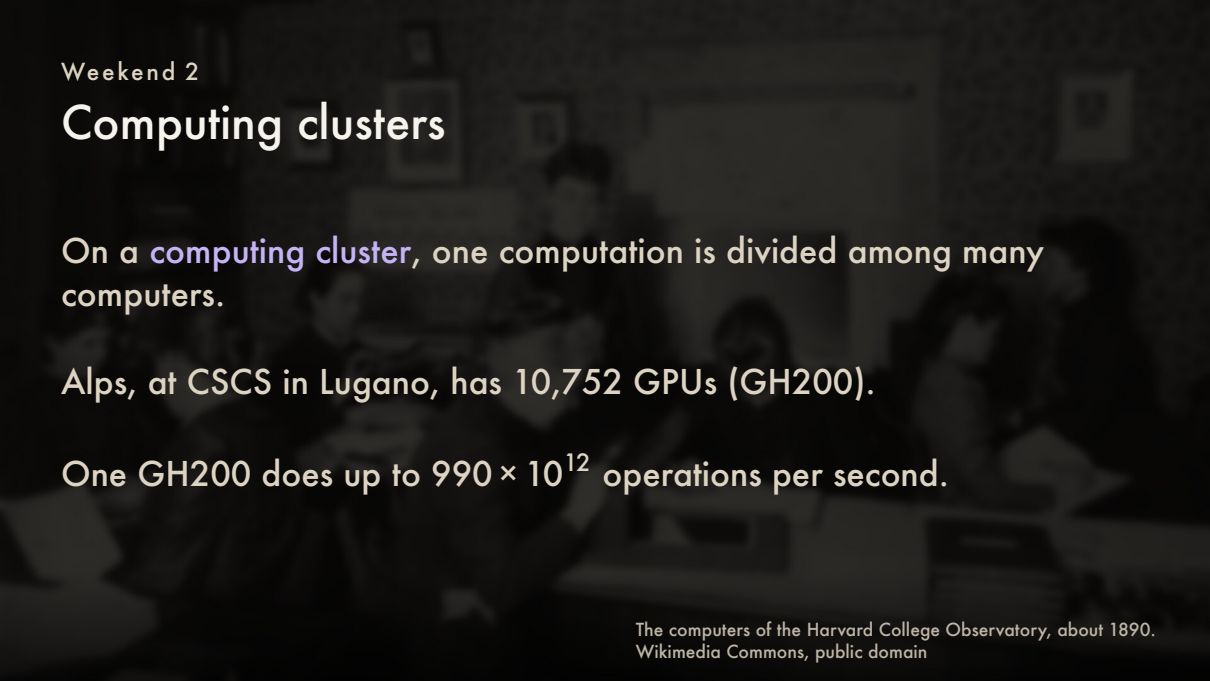
Weekend 2

# Computing clusters

On a computing cluster, one computation is divided among many computers.

Alps, at CSCS in Lugano, has 10,752 GPUs (GH200).

One GH200 does up to $990 \times 10^{12}$ operations per second.



The computers of the Harvard College Observatory, about 1890.
Wikimedia Commons, public domain

Weekend 2

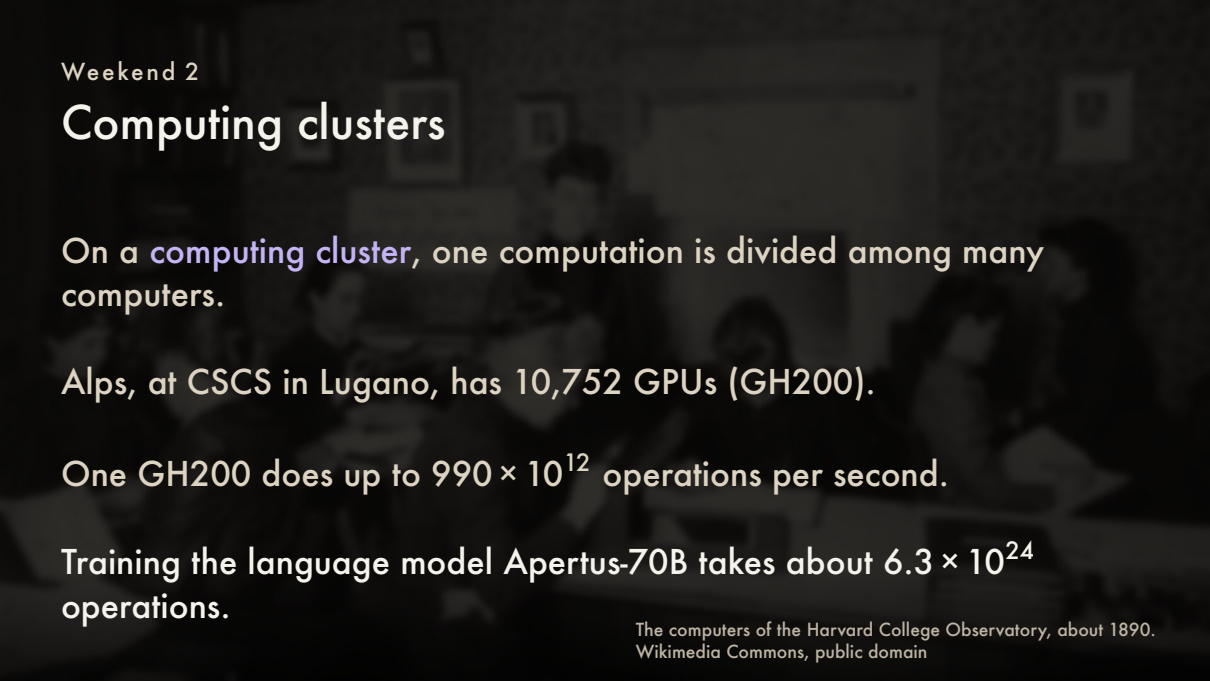
# Computing clusters

On a computing cluster, one computation is divided among many computers.

Alps, at CSCS in Lugano, has 10,752 GPUs (GH200).

One GH200 does up to $990 \times 10^{12}$ operations per second.

Training the language model Apertus-70B takes about $6.3 \times 10^{24}$ operations.

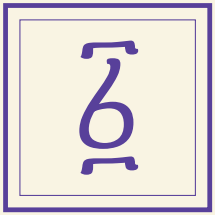


The computers of the Harvard College Observatory, about 1890.
Wikimedia Commons, public domain

# Business case

*Large-scale computing*

Carlos Cotrini
*AI in Industry, ETH Zurich*
*Friday 13 November 2026* 𓂀𓋹𓂀𓆑



*Section 1*

# LinkedIn’s ads

# LinkedIn's ads: the chance of a click

The chance that a member clicks an ad enters the ad auction, and the price depends on it.

Advertisers choose which clicks they pay for: some count likes or comments, others only visits to their website.

*Borisyuk and others (LinkedIn), KDD 2024.*

# LinkedIn's ads: the chance of a click

The chance that a member clicks an ad enters the ad auction, and the price depends on it.

Advertisers choose which clicks they pay for: some count likes or comments, others only visits to their website.

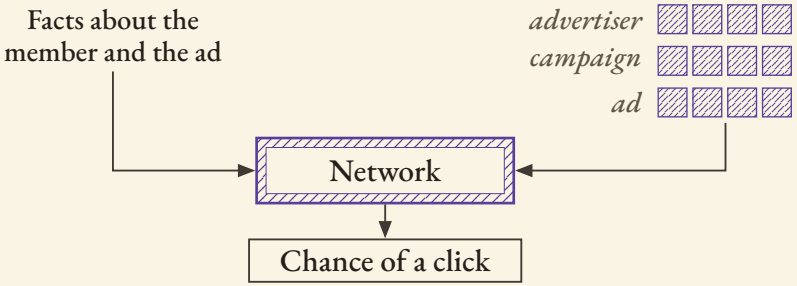
**Over 1 billion**

members

*Borisyuk and others (LinkedIn), KDD 2024.*

# Numbers for every advertiser, campaign and ad

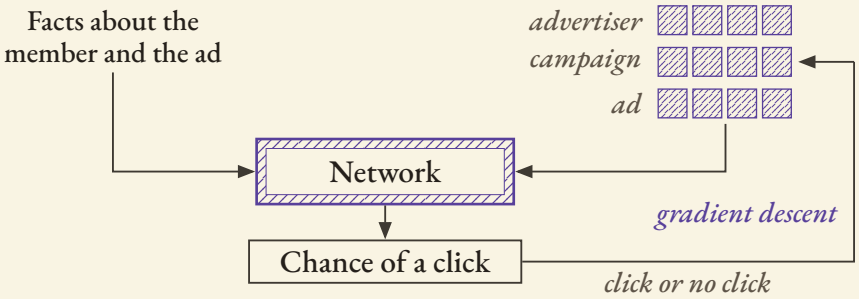
The matchmaker learned lists from ratings; here, from clicks.




*Borisyuk and others (LinkedIn), KDD 2024.*

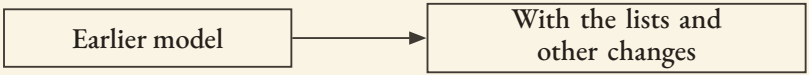
# Numbers for every advertiser, campaign and ad

The matchmaker learned lists from ratings; here, from clicks.



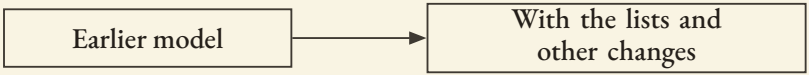

*Borisyuk and others (LinkedIn), KDD 2024.*

# The value: A/B tests




*Borisyuk and others (LinkedIn), KDD 2024.*

# The value: A/B tests



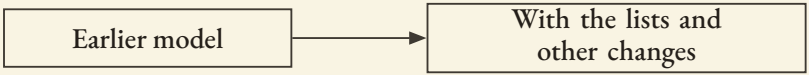

**+4.3%**

*click rate on ads, relative change*

LinkedIn put the new model into production.

*Borisyuk and others (LinkedIn), KDD 2024.*

# The value: A/B tests




**+4.3%**

*click rate on ads, relative change*

LinkedIn put the new model into production.

**−96%**

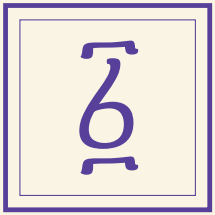
*training time in an offline test, when each new model builds on the earlier ones and learns from the newest half day of data*

*Borisyuk and others (LinkedIn), KDD 2024.*

# Computing clusters

*The cost of computation and supercomputers*

Carlos Cotrini
*AI in Industry, ETH Zurich*
*Friday 13 November 2026* 𓃵𓃵𓃵𓃵



*Section 1*

# One operation, and who does it

# A FLOP is one operation on two numbers

**One multiply accumulate**

# A FLOP is one operation on two numbers

**One multiply accumulate**

$a \times b$
one multiply

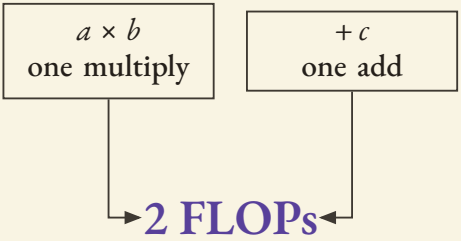
# A FLOP is one operation on two numbers

**One multiply accumulate**

| $a \times b$ | $+ c$ |
| --- | --- |
| one multiply | one add |

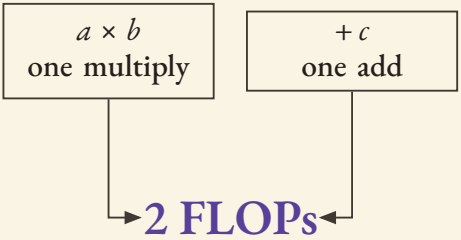
# A FLOP is one operation on two numbers

**One multiply accumulate**



# A FLOP is one operation on two numbers

**One multiply accumulate**

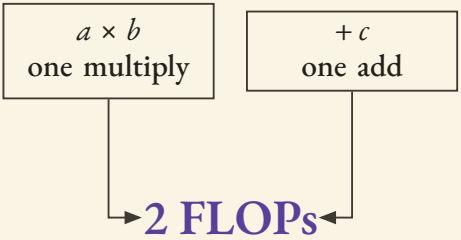



**Count, or rate**

**FLOP:** a count of operations
*how much work*

# A FLOP is one operation on two numbers

**One multiply accumulate**



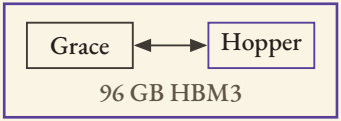

**Count, or rate**

**FLOP:** a count of operations
*how much work*

**FLOP/s:** operations per second
*how fast*

# One computer, then and now



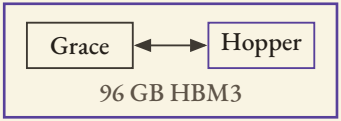



| | Human computer, 1949 | One GH200, 2024 |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

*Photograph: a computer room, NACA, 1949 (NASA Dryden, E49-54), public domain.*

# One computer, then and now



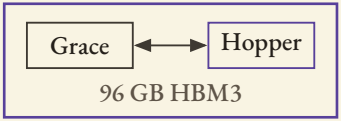



| | Human computer, 1949 | One GH200, 2024 |
|---|---|---|
| *Rate* | 1,000 additions a day | $990 \times 10^{12}$ FLOP/s |
| | | |
| | | |
| | | |

*Photograph: a computer room, NACA, 1949 (NASA Dryden, E49-54), public domain.*

# One computer, then and now



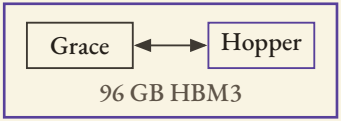



| | Human computer, 1949 | One GH200, 2024 |
|---|---|---|
| *Rate* | 1,000 additions a day | $990 \times 10^{12}$ FLOP/s |
| *Format* | Decimal, by hand | 16 bits per number |
| | | |
| | | |

*Photograph: a computer room, NACA, 1949 (NASA Dryden, E49-54), public domain.*

# One computer, then and now



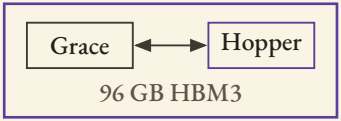



| | Human computer, 1949 | One GH200, 2024 |
|---|---|---|
| *Rate* | 1,000 additions a day | $990 \times 10^{12}$ FLOP/s |
| *Format* | Decimal, by hand | 16 bits per number |
| *Power* | About 100 W | 660 W on Alps |
| | | |

*Photograph: a computer room, NACA, 1949 (NASA Dryden, E49-54), public domain.*

# One computer, then and now



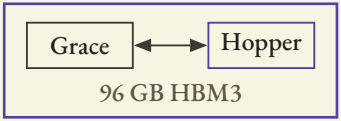



| | Human computer, 1949 | One GH200, 2024 |
|---|---|---|
| *Rate* | 1,000 additions a day | $990 \times 10^{12}$ FLOP/s |
| *Format* | Decimal, by hand | 16 bits per number |
| *Power* | About 100 W | 660 W on Alps |
| *Food* | **2,065 kcal a day**<br>*what one person eats* | **13,600 kcal a day**<br>*what six or seven people eat* |

*Photograph: a computer room, NACA, 1949 (NASA Dryden, E49-54), public domain.*

# One computer, then and now






| | Human computer, 1949 | One GH200, 2024 |
|---|---|---|
| *Rate* | 1,000 additions a day | $990 \times 10^{12}$ FLOP/s |
| *Format* | Decimal, by hand | 16 bits per number |
| *Power* | About 100 W | 660 W on Alps |
| *Food* | **2,065 kcal a day**<br>*what one person eats* | **13,600 kcal a day**<br>*what six or seven people eat* |

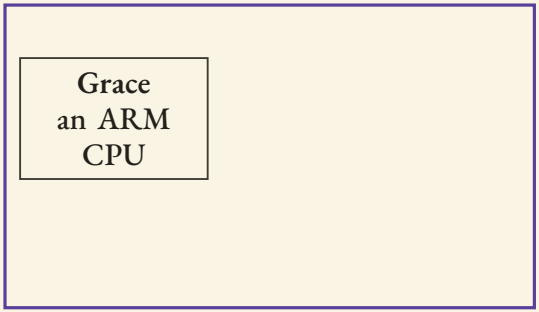
The clerk's rate is what the shop *delivered*. The chip's is what it *could* do.

*Photograph: a computer room, NACA, 1949 (NASA Dryden, E49-54), public domain.*

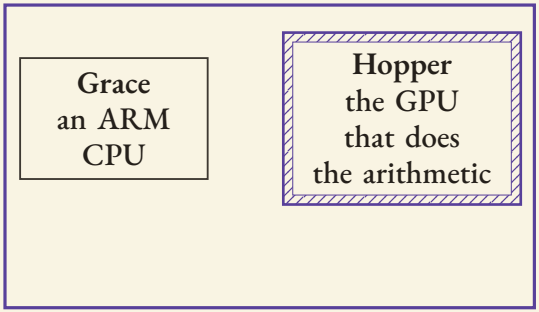
# A GH200 is a CPU and a GPU on one module



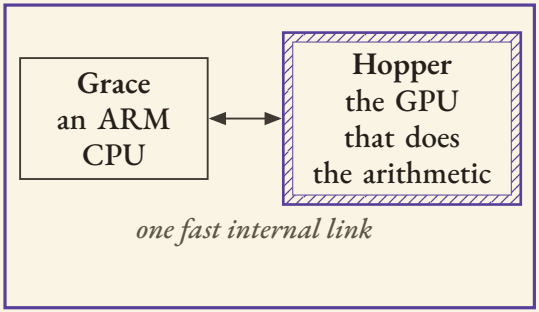
# A GH200 is a CPU and a GPU on one module



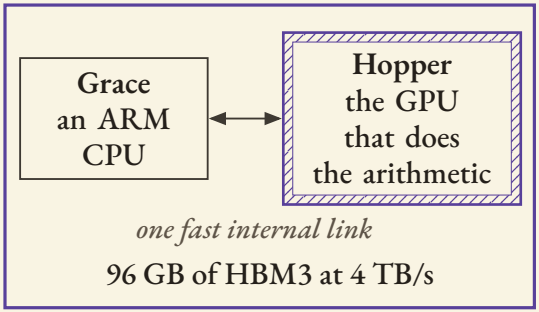
# A GH200 is a CPU and a GPU on one module



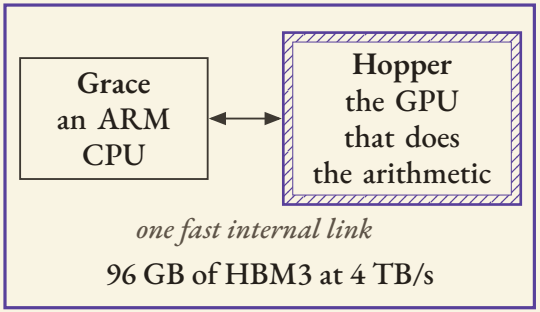
# A GH200 is a CPU and a GPU on one module



# A GH200 is a CPU and a GPU on one module

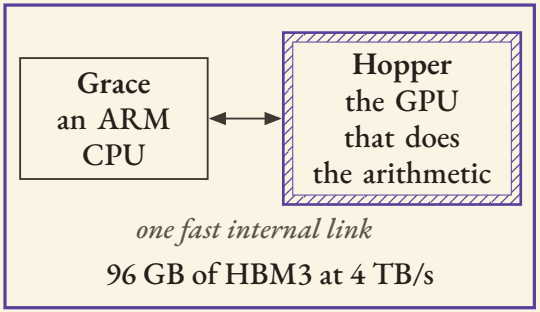


# A GH200 is a CPU and a GPU on one module




*The GPU computes. The CPU reads the data, runs Python, and keeps it fed*

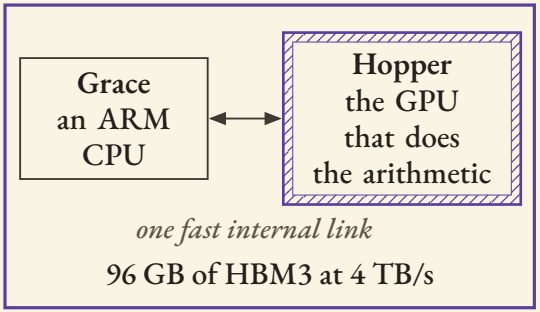
# A GH200 is a CPU and a GPU on one module




*The GPU computes. The CPU reads the data, runs Python, and keeps it fed*

*HBM3 is memory stacked on the GPU package: only 96 GB, but 4 TB/s*

# A GH200 is a CPU and a GPU on one module




- *The GPU computes. The CPU reads the data, runs Python, and keeps it fed*
- *HBM3 is memory stacked on the GPU package: only 96 GB, but 4 TB/s*
- *Between them 900 GB/s, about seven times what a plug-in card gets*

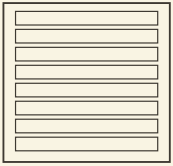
# Alps is an HPE Cray EX254n

The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*

# Alps is an HPE Cray EX254n

The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*

**HPE Cray EX, the platform**

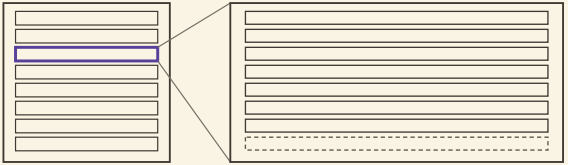


*one of 24 cabinets: 8 chassis*

# Alps is an HPE Cray EX254n

The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*

**HPE Cray EX, the platform**

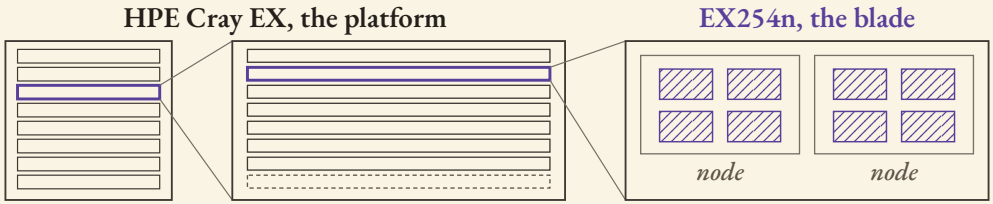


*one of 24 cabinets: 8 chassis*

*one chassis: 8 blade slots, 7 used at Alps, for more power per GPU*

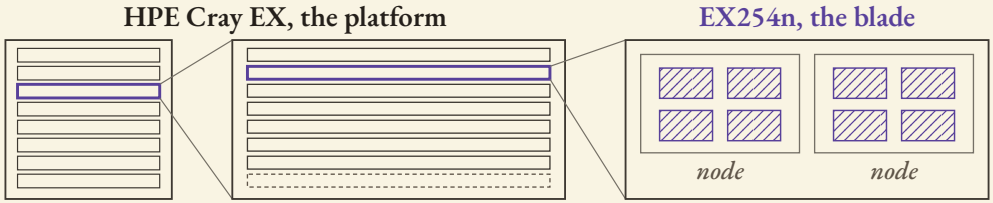
# Alps is an HPE Cray EX254n

The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*



# Alps is an HPE Cray EX254n

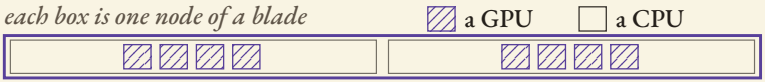
The TOP500 entry reads: *Alps, HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS*



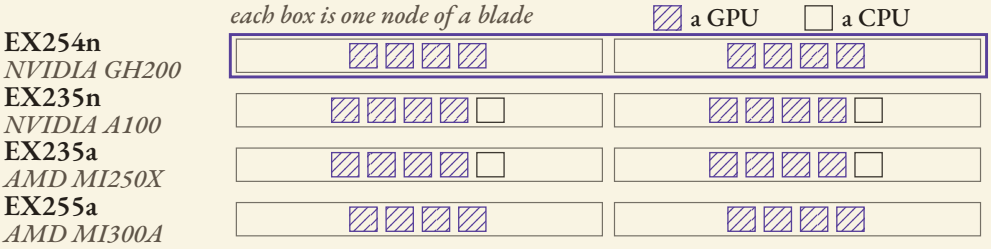

The platform: cabinets, liquid cooling, power and the network.

# One platform, several blade types

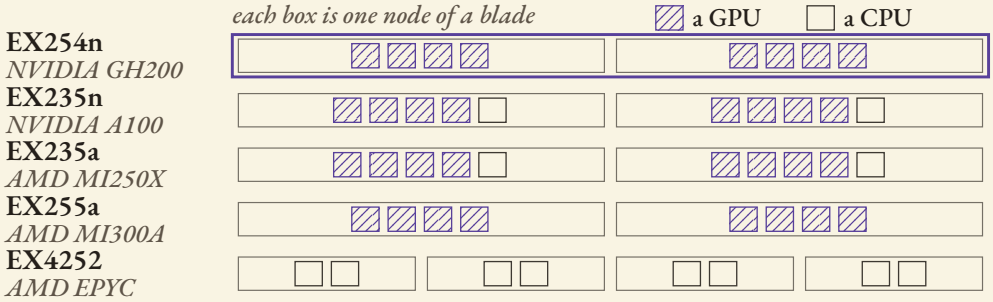
**EX254n**
*NVIDIA GH200*



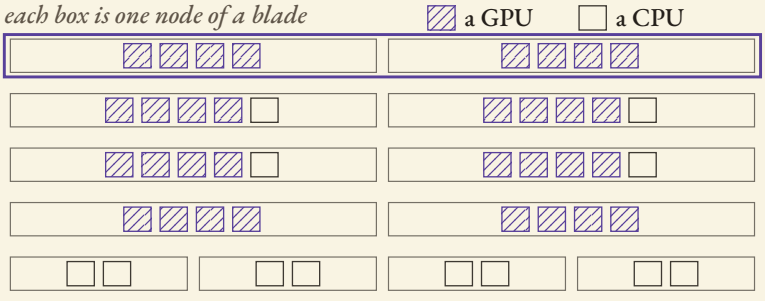
# One platform, several blade types



# One platform, several blade types

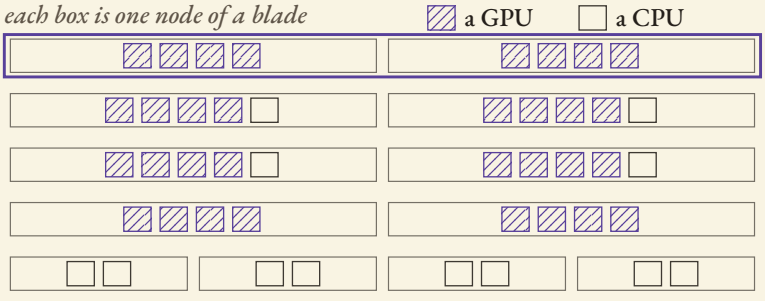


# One platform, several blade types




*Every name here that ends in n carries NVIDIA GPUs.*

# One platform, several blade types




*Every name here that ends in n carries NVIDIA GPUs.*
The blade choice is what makes Alps a machine for AI rather than a conventional CPU cluster.

# The atelier against Alps

| A collection of computers | Rate | Power |
|---|---|---|
| Prony's atelier, 1795<br>twenty people | 0.694 additions/s | about 2 kW |
| | | |

# The atelier against Alps

| A collection of computers | Rate | Power |
|---|---|---|
| Prony's atelier, 1795<br>twenty people | 0.694 additions/s | about 2 kW |
| Alps, 2024<br>10,752 modules | $10.64 \times 10^{18}$ FLOP/s | 7.1 MW |

# The atelier against Alps

| A collection of computers | Rate | Power |
|---|---|---|
| Prony's atelier, 1795<br>twenty people | 0.694 additions/s | about 2 kW |
| Alps, 2024<br>10,752 modules | $10.64 \times 10^{18}$ FLOP/s | 7.1 MW |

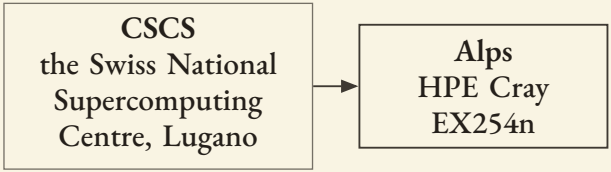
**Fifteen billion billion times the rate,**
**on three and a half thousand times the power**

# CSCS and Alps, the machine behind Apertus

**CSCS**
the Swiss National Supercomputing Centre, Lugano

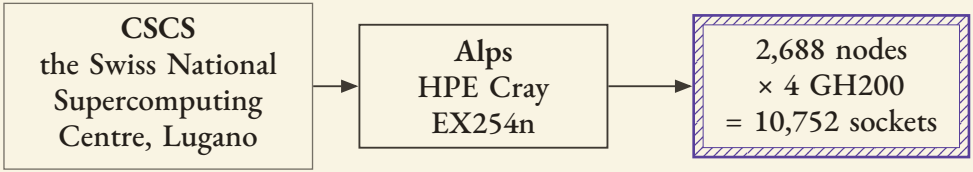
*Machine: cscs.ch/computers/alps and top500.org/system/180259. Model: arXiv 2509.14233.*

# CSCS and Alps, the machine behind Apertus



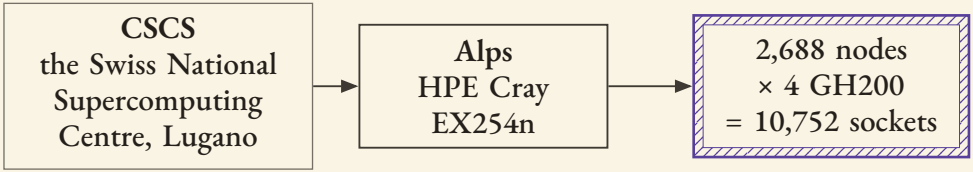

*Machine: cscs.ch/computers/alps and top500.org/system/180259. Model: arXiv 2509.14233.*

# CSCS and Alps, the machine behind Apertus




*Machine: cscs.ch/computers/alps and top500.org/system/180259. Model: arXiv 2509.14233.*

# CSCS and Alps, the machine behind Apertus

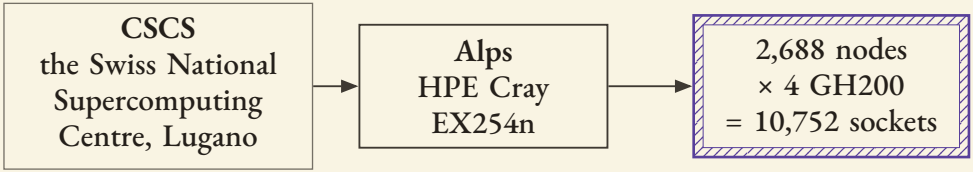



**10.6 EFLOP/s** $= 10.6 \times 10^{18}$ **FLOP/s**

*all 10,752 GH200 together*

*Machine: cscs.ch/computers/alps and top500.org/system/180259. Model: arXiv 2509.14233.*

# CSCS and Alps, the machine behind Apertus




**10.6 EFLOP/s** $= 10.6 \times 10^{18}$ **FLOP/s**

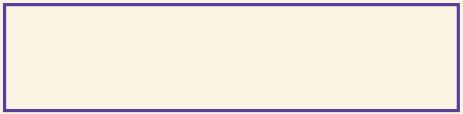
*all 10,752 GH200 together*

Apertus, the open language model of EPFL, ETH Zurich and CSCS, was trained on Alps.

*Machine: cscs.ch/computers/alps and top500.org/system/180259. Model: arXiv 2509.14233.*

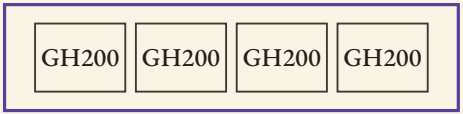
# Nodes and sockets

One node



# Nodes and sockets

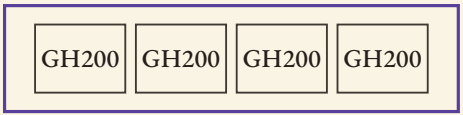
One node




*One operating system, one address space*

# Nodes and sockets

One node



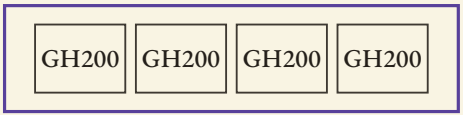

*One operating system, one address space*

**Socket** one processor module

**Node** one independent computer

# Nodes and sockets

One node




*One operating system, one address space*

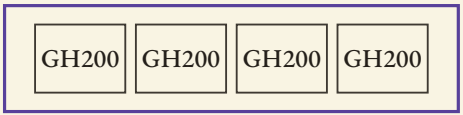
**Socket** one processor module

**Node** one independent computer

2,688 nodes × 4 modules = 10,752 sockets

# Nodes and sockets

One node




*One operating system, one address space*

**Socket** one processor module

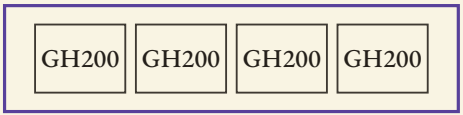
**Node** one independent computer

2,688 nodes × 4 modules = 10,752 sockets

Socket to socket, inside the node: **150 GB/s**

# Nodes and sockets

One node




*One operating system, one address space*

**Socket** one processor module

**Node** one independent computer

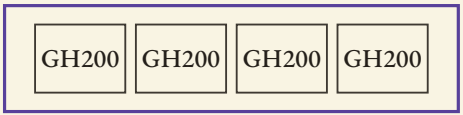
2,688 nodes × 4 modules = 10,752 sockets

Socket to socket, inside the node: **150 GB/s**

Between nodes: **25 GB/s** per GPU, six times less

# Nodes and sockets

One node




*One operating system, one address space*

**Socket** one processor module

**Node** one independent computer

2,688 nodes × 4 modules = 10,752 sockets

Socket to socket, inside the node: **150 GB/s**

Between nodes: **25 GB/s per GPU, six times less**

The node boundary is where the bandwidth falls off.

# One second of a GH200, in three registers

| The same one second of arithmetic | Time |
|---|---|
| The mean age of all 8.2 billion people alive, one addition each | 8.3 microseconds |
| | |
| | |

*ResNet-50: He and others, arXiv 1512.03385: 3.8 billion multiply-adds, so* $7.6 \times 10^9$ *FLOP per image. A peak count.*

# One second of a GH200, in three registers

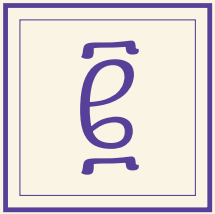
| The same one second of arithmetic | Time |
|---|---|
| The mean age of all 8.2 billion people alive, one addition each | 8.3 microseconds |
| 130,000 photographs classified by ResNet-50, each 224 by 224 pixels, or 0.05 megapixels | one second |
| | |

*ResNet-50: He and others, arXiv 1512.03385: 3.8 billion multiply-adds, so* $7.6 \times 10^9$ *FLOP per image. A peak count.*

# One second of a GH200, in three registers

| The same one second of arithmetic | Time |
|---|---|
| The mean age of all 8.2 billion people alive, one addition each | 8.3 microseconds |
| 130,000 photographs classified by ResNet-50, each 224 by 224 pixels, or 0.05 megapixels | one second |
| Prony's atelier doing that same one second of work | 45 to 136 million years |

*ResNet-50: He and others, arXiv 1512.03385: 3.8 billion multiply-adds, so* $7.6 \times 10^9$ *FLOP per image. A peak count.*



*Section 2*

# What the headline rate hides

# Capacity and bandwidth are different numbers

**96 GB**
how much fits

# Capacity and bandwidth are different numbers

**96 GB**
how much fits

**4 TB/s**
how fast you can move it

# Capacity and bandwidth are different numbers

**96 GB**
how much fits

**4 TB/s**
how fast you can move it

Decides whether the weights and the optimiser state fit on one device at all

# Capacity and bandwidth are different numbers

**96 GB**
how much fits

Decides whether the weights and the optimiser state fit on one device at all

**4 TB/s**
how fast you can move it

Decides how quickly the arithmetic units can be fed once the data is there

# Capacity and bandwidth are different numbers

**96 GB**
how much fits

Decides whether the weights and the optimiser state fit on one device at all

**4 TB/s**
how fast you can move it

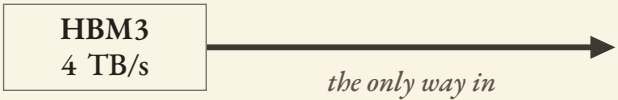
Decides how quickly the arithmetic units can be fed once the data is there

96 GB is less than an ordinary server holds. That is the trade: HBM is stacked on the package and wired short and wide, which is what buys the speed.

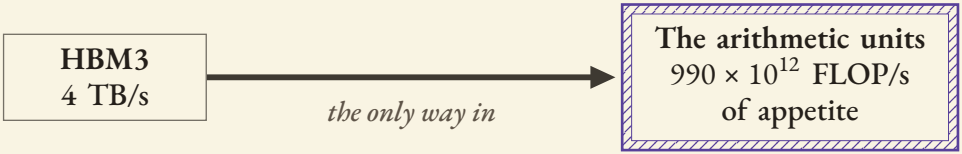
# Two hundred and forty six operations per byte

**HBM3**
4 TB/s

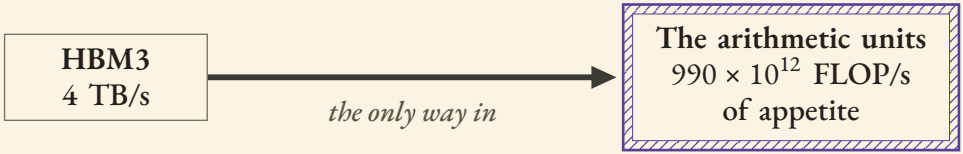
# Two hundred and forty six operations per byte



# Two hundred and forty six operations per byte

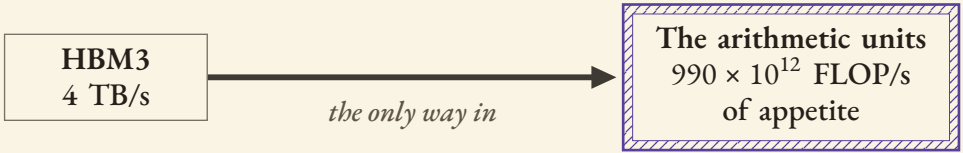


# Two hundred and forty six operations per byte




$$\frac{990 \times 10^{12}\ \text{FLOP/s}}{4.023 \times 10^{12}\ \text{bytes/s}} = 246\ \text{FLOP per byte}$$

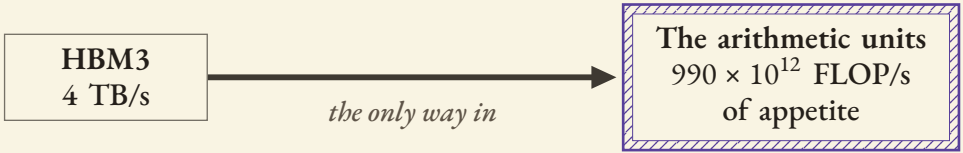
# Two hundred and forty six operations per byte




$$\frac{990 \times 10^{12}\ \text{FLOP/s}}{4.023 \times 10^{12}\ \text{bytes/s}} = 246\ \text{FLOP per byte}$$

*A rate over a rate: the seconds cancel, and what is left is operations per byte*

# Two hundred and forty six operations per byte

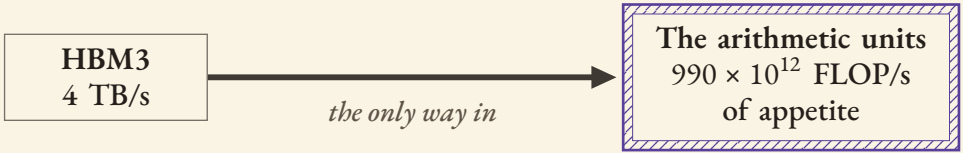



$$\frac{990 \times 10^{12}\ \text{FLOP/s}}{4.023 \times 10^{12}\ \text{bytes/s}} = 246\ \text{FLOP per byte}$$

*A rate over a rate: the seconds cancel, and what is left is operations per byte*

So unless the computation reuses each byte about that many times, the units wait. A large matrix multiply does reuse. Many other operations do not.

# Two hundred and forty six operations per byte

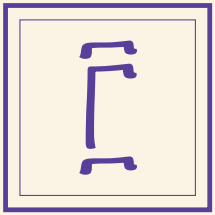



$$\frac{990 \times 10^{12}\ \text{FLOP/s}}{4.023 \times 10^{12}\ \text{bytes/s}} = 246\ \text{FLOP per byte}$$

*A rate over a rate: the seconds cancel, and what is left is operations per byte*

So unless the computation reuses each byte about that many times, the units wait. A large matrix multiply does reuse. Many other operations do not.

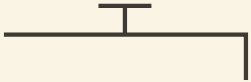
This is one of the reasons a real run reaches 21 to 46 percent of peak.



*Section 3*

# The machine, and what it draws

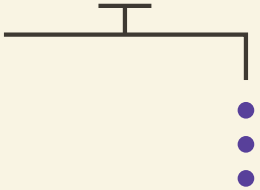
# A watt is a rate, a kilowatt hour is an amount



**660 W**

How fast energy arrives

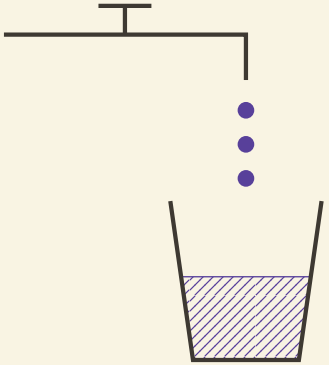
# A watt is a rate, a kilowatt hour is an amount



**660 W**

How fast energy arrives

# A watt is a rate, a kilowatt hour is an amount



**660 W**

How fast energy arrives

**0.66 kWh** *after one hour*

How much arrived

# Watch the slash

**km/h** The slash divides, so it is a speed.

# Watch the slash

| | |
|---|---|
| **km/h** | The slash divides, so it is a speed. |
| **kWh** | No slash. The hour **multiplies**, so it is an amount. |

# Watch the slash

**km/h** The slash divides, so it is a speed.

**kWh** No slash. The hour **multiplies**, so it is an amount.

Longer gives you *more*, so the hour cannot be dividing.

# Watch the slash

**km/h** The slash divides, so it is a speed.

**kWh** No slash. The hour **multiplies**, so it is an amount.

Longer gives you *more*, so the hour cannot be dividing.

---

rate × time = amount

# Watch the slash

**km/h** The slash divides, so it is a speed.

**kWh** No slash. The hour **multiplies**, so it is an amount.

Longer gives you *more*, so the hour cannot be dividing.

| rate | × | time | = | amount |
|---|---|---|---|---|
| FLOP/s | × | seconds | = | FLOP |

# Watch the slash

**km/h** The slash divides, so it is a speed.

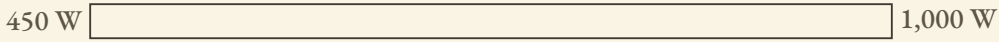
**kWh** No slash. The hour **multiplies**, so it is an amount.

Longer gives you *more*, so the hour cannot be dividing.

| rate | × | time | = | amount |
|---|---|---|---|---|
| FLOP/s | × | seconds | = | FLOP |
| watts | × | hours | = | watt hours |

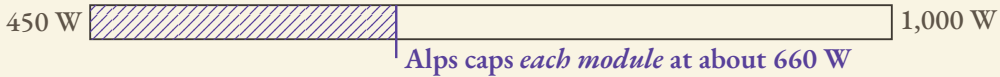
# A wattage becomes a bill in two multiplications

**One module is just a band**



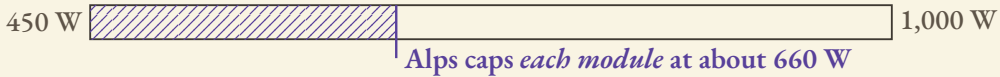
# A wattage becomes a bill in two multiplications

**One module is just a band**



# A wattage becomes a bill in two multiplications

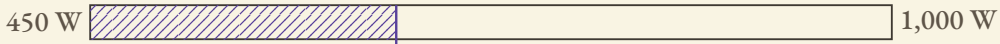
**One module is just a band**




*The operator chooses: near 1,000 W is faster, lower saves energy and cooling*

# A wattage becomes a bill in two multiplications

**One module is just a band**



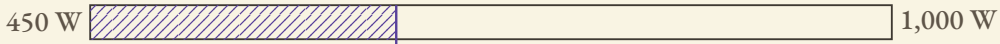

**Alps caps *each module* at about 660 W**

*The operator chooses: near 1,000 W is faster, lower saves energy and cooling*

660 W
for one hour

# A wattage becomes a bill in two multiplications

**One module is just a band**



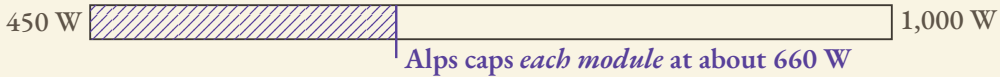

**Alps caps *each module* at about 660 W**

*The operator chooses: near 1,000 W is faster, lower saves energy and cooling*

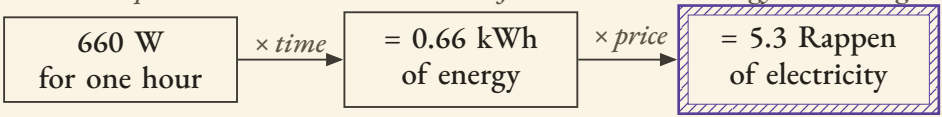
660 W for one hour → *× time* → = 0.66 kWh of energy

# A wattage becomes a bill in two multiplications

**One module is just a band**



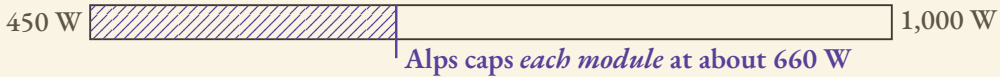

*The operator chooses: near 1,000 W is faster, lower saves energy and cooling*



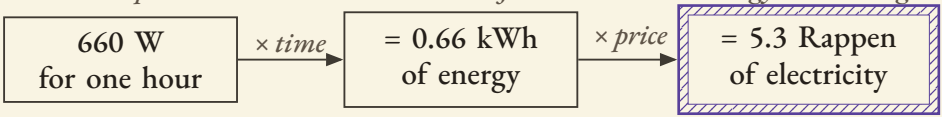

*450 to 1,000 W: GH200 datasheet. 660 W: the Alps cap, docs.cscs.ch/running/slurm. 8 Rappen per kWh assumed (ElCom 2026, large consumers 6.75 to 9; households pay 27.7). Check: 10,752 × 660 W is 7,096 kW, against the 7,124 kW the TOP500 measured.*

# A wattage becomes a bill in two multiplications

**One module is just a band**




*The operator chooses: near 1,000 W is faster, lower saves energy and cooling*

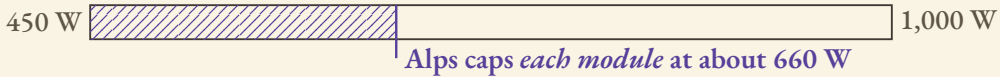



That is one module. All 10,752 is 7.1 megawatts, about 570 francs an hour.

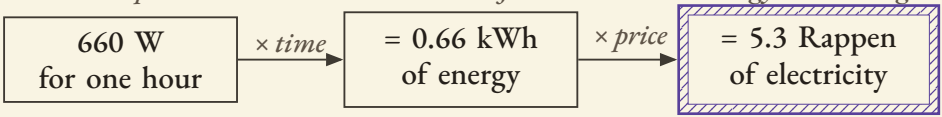
*450 to 1,000 W: GH200 datasheet. 660 W: the Alps cap, docs.cscs.ch/running/slurm. 8 Rappen per kWh assumed (ElCom 2026, large consumers 6.75 to 9; households pay 27.7). Check: 10,752 × 660 W is 7,096 kW, against the 7,124 kW the TOP500 measured.*

# A wattage becomes a bill in two multiplications

**One module is just a band**



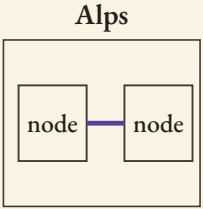

*The operator chooses: near 1,000 W is faster, lower saves energy and cooling*



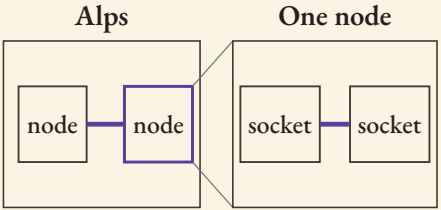

That is one module. All 10,752 is 7.1 megawatts, about 570 francs an hour.
The whole Apertus run is 3.96 GWh, about 0.3 million francs.

*450 to 1,000 W: GH200 datasheet. 660 W: the Alps cap, docs.cscs.ch/running/slurm. 8 Rappen per kWh assumed (ElCom 2026, large consumers 6.75 to 9; households pay 27.7). Check: 10,752 × 660 W is 7,096 kW, against the 7,124 kW the TOP500 measured.*

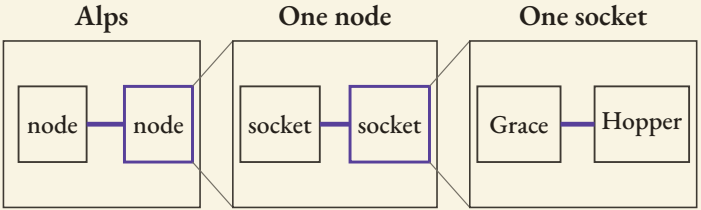
# Four links, and each step down costs you



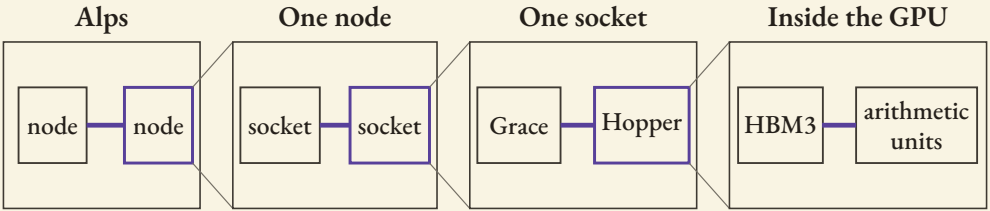
# Four links, and each step down costs you



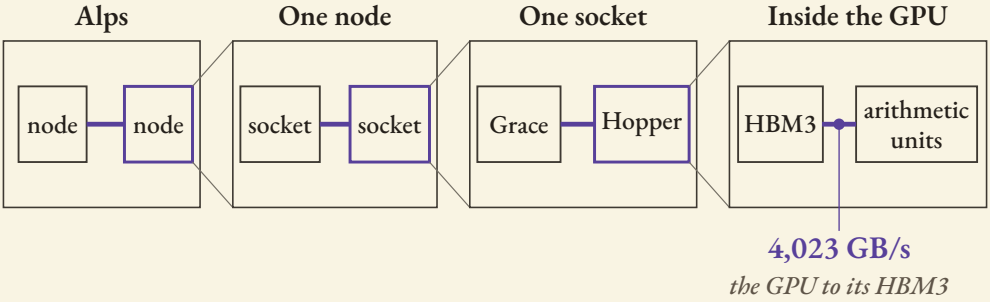
# Four links, and each step down costs you



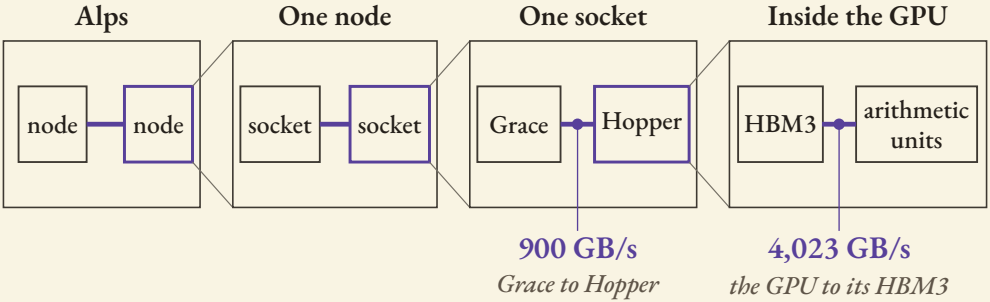
# Four links, and each step down costs you



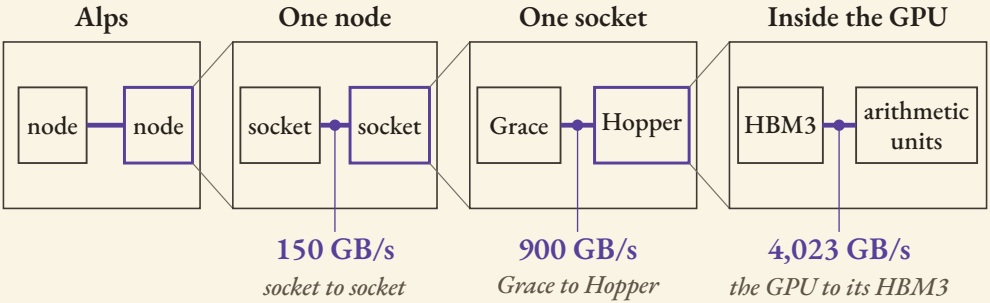
# Four links, and each step down costs you



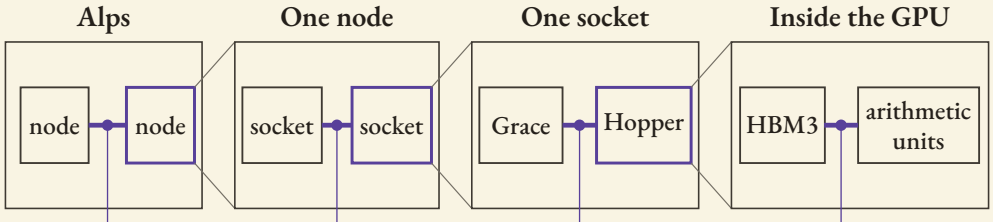
# Four links, and each step down costs you



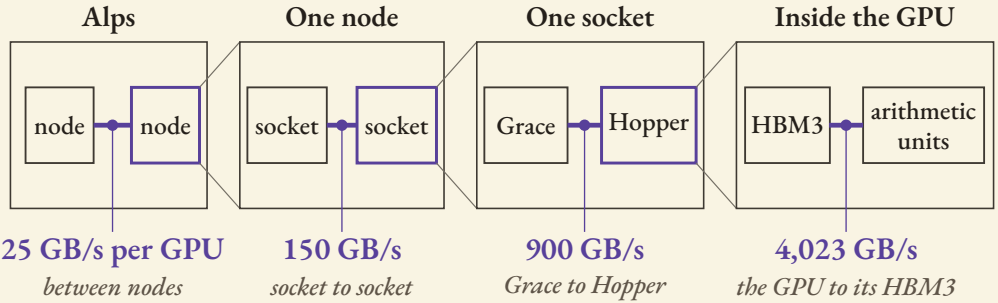
# Four links, and each step down costs you



# Four links, and each step down costs you

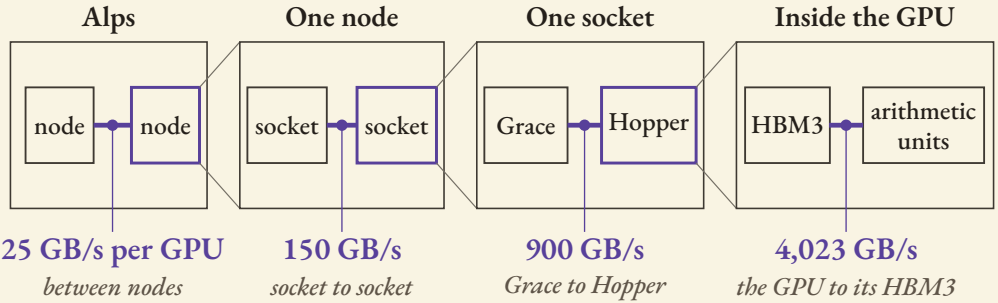


# Four links, and each step down costs you



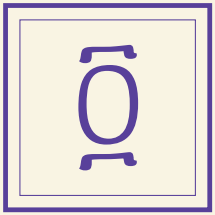

Each step down costs about a factor of five. End to end, 161 times.

# Four links, and each step down costs you




**Each step down costs about a factor of five. End to end, 161 times.**

So you keep the data at the fast end and cross the slow links as rarely as you can.



*Section 4*

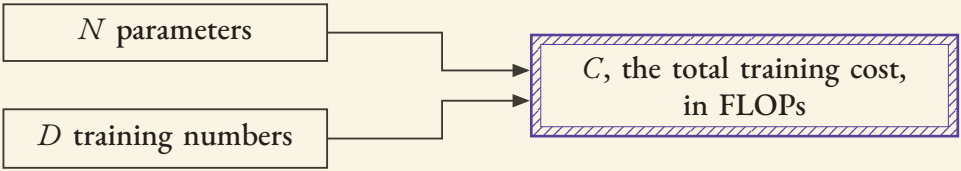
# From a model to GPU hours

# Two numbers price a training run

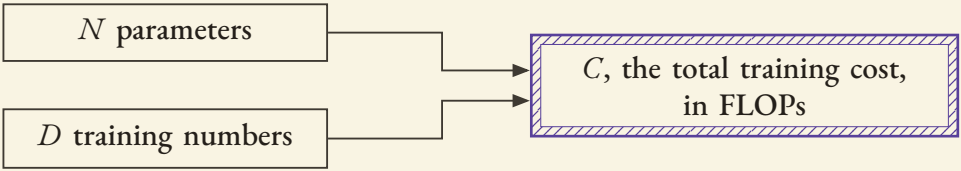
$N$ parameters

$D$ training numbers

# Two numbers price a training run

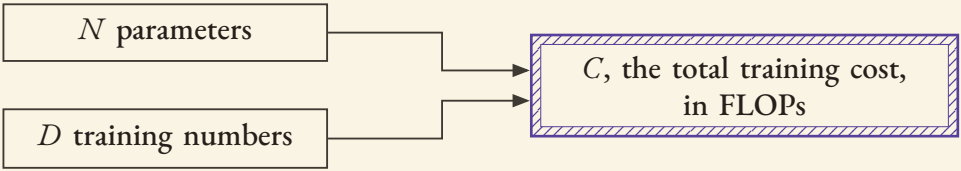


# Two numbers price a training run



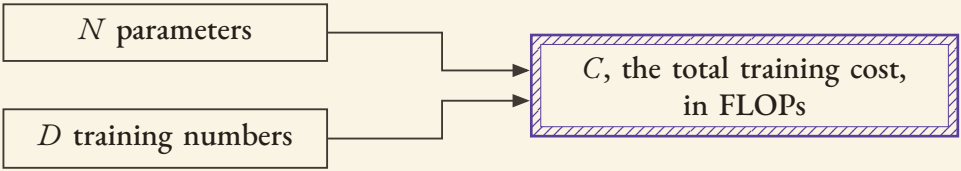

$$C = \underbrace{N}_{\text{parameters}}$$

# Two numbers price a training run



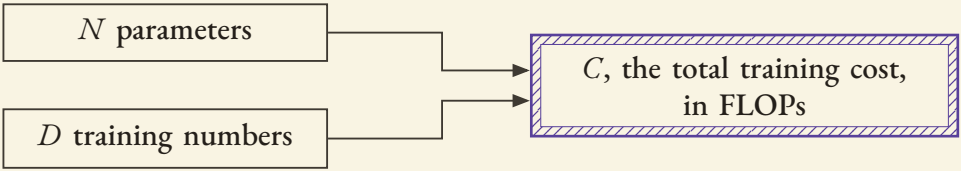

$$C = \underbrace{N}_{\text{parameters}} \times \underbrace{D}_{\text{training numbers}}$$

# Two numbers price a training run




$$C = \underbrace{6}_{\text{per parameter, per training number}} \times \underbrace{N}_{\text{parameters}} \times \underbrace{D}_{\text{training numbers}}$$

# Two numbers price a training run




$$C = \underbrace{6}_{\text{per parameter, per training number}} \times \underbrace{N}_{\text{parameters}} \times \underbrace{D}_{\text{training numbers}}$$

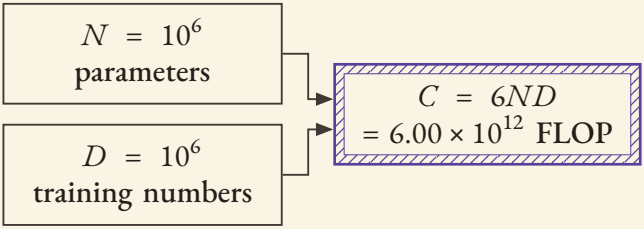
*Kaplan and others, 2020, section 2.1. Here the 6 is a rule of thumb.*

# A million by a million is two minutes

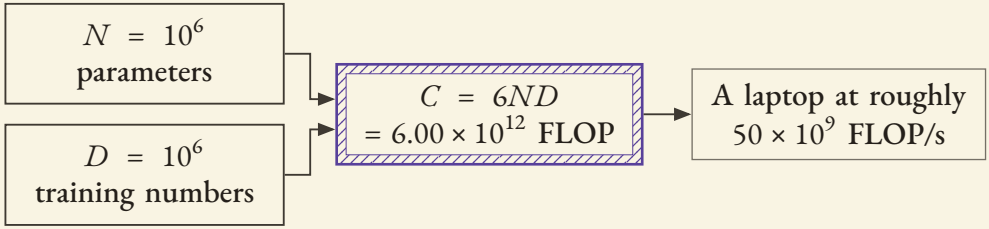
$N = 10^6$
parameters

$D = 10^6$
training numbers

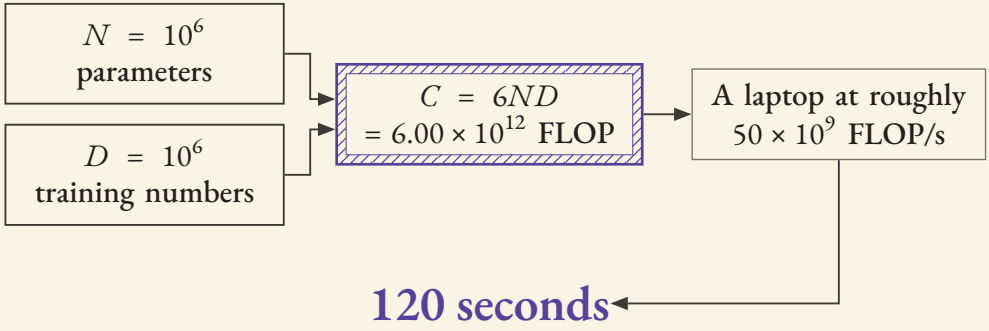
# A million by a million is two minutes



# A million by a million is two minutes



# A million by a million is two minutes



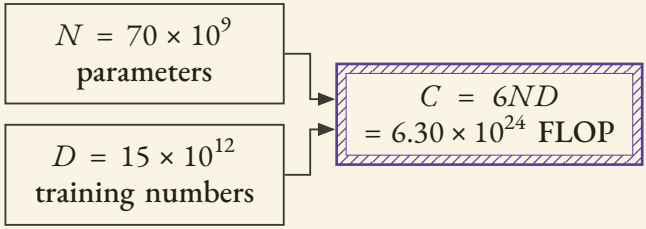
# The same formula on Apertus-70B

$N = 70 \times 10^9$
parameters

$D = 15 \times 10^{12}$
training numbers

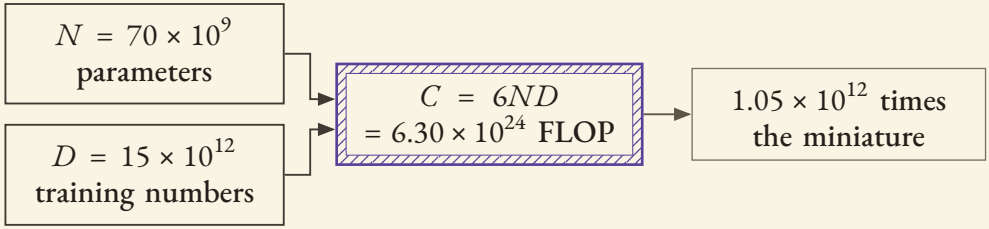
*Model: arXiv 2509.14233.*

# The same formula on Apertus-70B



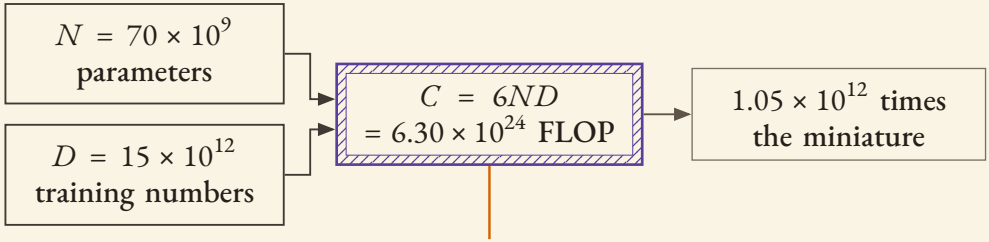

*Model: arXiv 2509.14233.*

# The same formula on Apertus-70B




*Model: arXiv 2509.14233.*

# The same formula on Apertus-70B




*Model: arXiv 2509.14233.*

# From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion training numbers took over 6 million GPU hours, on up to 4,096 GPUs.

*Run: arXiv 2604.12973.*

# From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion training numbers took over 6 million GPU hours, on up to 4,096 GPUs.

4,096 GPUs flat out
for 61 days

*Run: arXiv 2604.12973.*

# From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion training numbers took over 6 million GPU hours, on up to 4,096 GPUs.

| | |
|---|---|
| 4,096 GPUs flat out for 61 days | It spanned about 91 days, so 2,747 on average |

*Run: arXiv 2604.12973.*

# From one GPU hour to the whole run

The run: training Apertus-70B on 15 trillion training numbers took over 6 million GPU hours, on up to 4,096 GPUs.

| 4,096 GPUs flat out for **61 days** | It spanned about 91 days, so **2,747** on average |
|---|---|

*The gap is ramp up, restarts and smaller phases: a campaign rarely runs at full width without a break*

*Run: arXiv 2604.12973.*

# From one GPU hour to the whole run

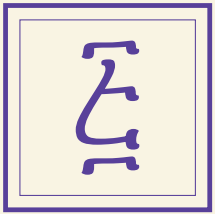
The run: training Apertus-70B on 15 trillion training numbers took over 6 million GPU hours, on up to 4,096 GPUs.

| 4,096 GPUs flat out for **61 days** | It spanned about 91 days, so **2,747** on average |
|---|---|

*The gap is ramp up, restarts and smaller phases: a campaign rarely runs at full width without a break*

6 million GPU hours × 660 W = **3.96 GWh**
× 8 Rappen = **0.32 million francs**

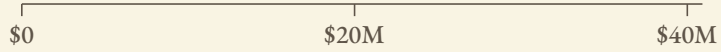
*Run: arXiv 2604.12973.*



*Section 5*

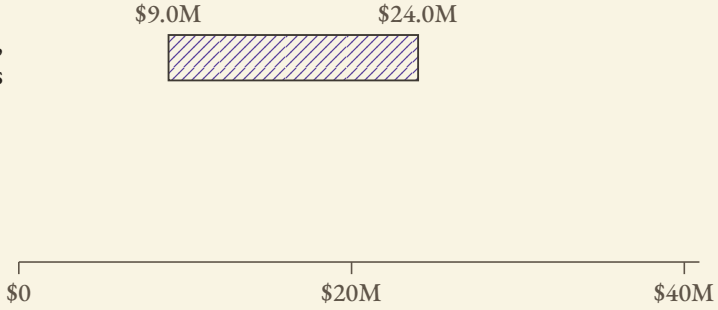
# Dollars and energy

# Six million hours, priced at cloud rates

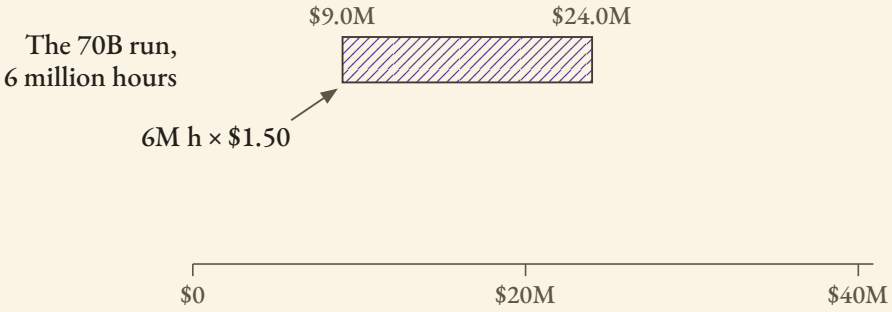


# Six million hours, priced at cloud rates

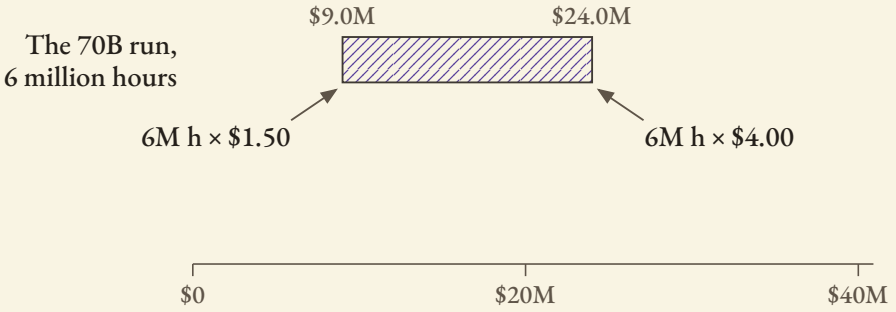
The 70B run,
6 million hours



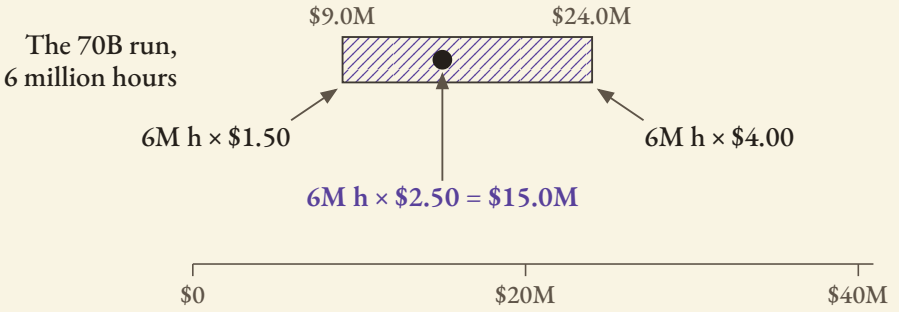
# Six million hours, priced at cloud rates



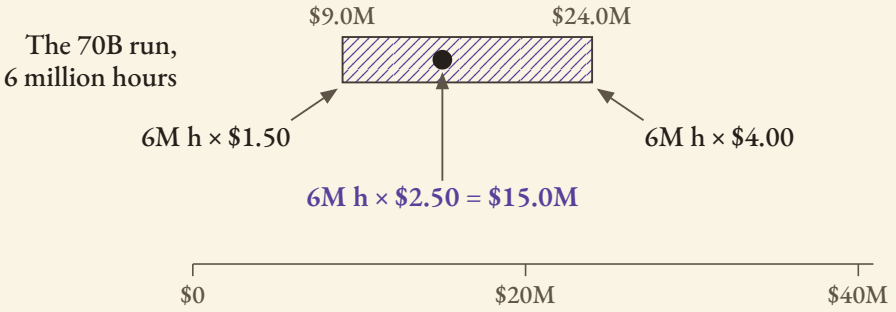
# Six million hours, priced at cloud rates



# Six million hours, priced at cloud rates



# Six million hours, priced at cloud rates




*The $1.50 to $4.00 band is an H100 rental survey, November 2025.*

# The same hours, in gigawatt hours

8.85 million GPU hours
90 days × 4,096 GPUs

# The same hours, in gigawatt hours

8.85 million GPU hours
90 days × 4,096 GPUs

×

560 W
per module

# The same hours, in gigawatt hours

8.85 million GPU hours
90 days × 4,096 GPUs

×

560 W
per module

=

**4.95 GWh**

# The same hours, in gigawatt hours

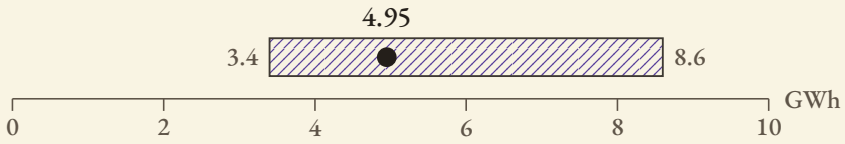
8.85 million GPU hours
90 days × 4,096 GPUs

×

560 W
per module

=

**4.95 GWh**



# The same hours, in gigawatt hours

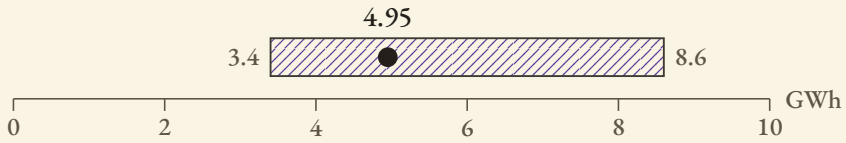
8.85 million GPU hours
90 days × 4,096 GPUs

×

560 W
per module

=

**4.95 GWh**




About 1,100 Swiss households for a year, at 4,500 kWh each.

# The same hours, in gigawatt hours

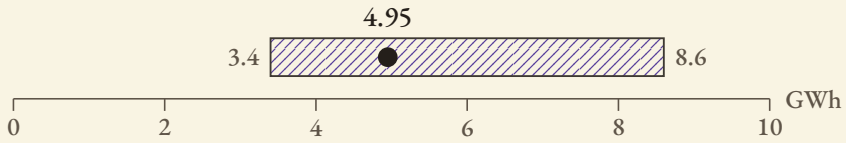
8.85 million GPU hours
90 days × 4,096 GPUs

×

560 W
per module

=

**4.95 GWh**




About 1,100 Swiss households for a year, at 4,500 kWh each.

*Range 750 to 1,900 households, over four sourced module figures: 560 W, tech report, arXiv 2509.14233; 660 W, CSCS cap, docs.cscs.ch/running/slurm; 855 W, from the TOP500 run; datasheet band 450 to 1,000 W. Household 4,500 kWh a year is ElCom's H4 profile. CSCS publish a PUE below 1.2, cooling from Lake Lugano (cscs.ch).*